

```

from collections import deque

# Graph represented as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

def bfs(graph, start):
    visited = set()      # To keep track of visited nodes
    queue = deque()      # Create a queue

    visited.add(start)
    queue.append(start)

    while queue:
        vertex = queue.popleft() # Remove the front node
        print(vertex, end=" ")

        # Visit all adjacent nodes
        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

# Driver code
print("BFS Traversal:")
bfs(graph, 'A')

```

```

IDLE Shell 3.10.0
File Edit Shell Debug Options Window Help
Python 3.10.0 (tags/v3.10.0:b494f59, Oct 4 2021, 19:00:18) [MSC v.1929 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
===== RESTART: C:/Users/hi/OneDrive/Desktop/AI/expp.py =====
BFS Traversal:
A B C D E F G
>>>

expp.py - C:/Users/hi/OneDrive/Desktop/AI/expp.py (3.10.0)
File Edit Format Run Options Window Help
from collections import deque

# Graph represented as an adjacency list
graph = {
    'A': ['B', 'C'],
    'B': ['D', 'E'],
    'C': ['F', 'G'],
    'D': [],
    'E': [],
    'F': [],
    'G': []
}

def bfs(graph, start):
    visited = set()          # To keep track of visited nodes
    queue = deque()         # Create a queue

    visited.add(start)
    queue.append(start)

    while queue:
        vertex = queue.popleft() # Remove the front node
        print(vertex, end=" ")

        # Visit all adjacent nodes
        for neighbor in graph[vertex]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)

# Driver code
print("BFS Traversal:")
bfs(graph, 'A')

```